LAPORAN AKHIR

PEMROGRAMAN BERORIENTASI OBJEK



Oleh :

[Nama Lengkap]

NIM 2511531013

MATA KULIAH PEMROGRAMAN BERORIENTASI OBJEK

DOSEN PENGAMPU : [Nama Dosen Pengampu]

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Akhir Praktikum Pemrograman Berorientasi Objek ini dengan baik. Laporan ini disusun sebagai dokumentasi seluruh kegiatan praktikum dari praktikum 1 sampai praktikum 5, sekaligus sebagai media belajar dalam memahami konsep dasar pemrograman berorientasi objek menggunakan bahasa Java.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing selama pelaksanaan praktikum, serta kepada semua pihak yang telah membantu dalam penyusunan laporan ini. Penulis menyadari laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat diharapkan. Semoga laporan ini bermanfaat bagi pembaca.

Padang, Oktober 2026

Penulis

DAFTAR ISI

BAB 1 4
BAB 2 5
A. PRAKTIKUM 1 5
B. PRAKTIKUM 2 5
C. PRAKTIKUM 3 6
D. PRAKTIKUM 4 7
E. PRAKTIKUM 5 8
F. GITHUB DAN VIDEO PENJELASAN 11

BAB 1
PENDAHULUAN

1. Latar Belakang

Pemrograman berorientasi objek (PBO) adalah paradigma pemrograman yang menyusun program dari kumpulan objek. Setiap objek memiliki data (properti) dan perilaku (method), serta dapat berinteraksi dengan objek lain. Dengan cara ini, masalah yang kompleks dapat dipecah menjadi bagian-bagian kecil yang mudah dikelola, digunakan ulang, dan dikembangkan.

Dalam praktikum ini, konsep PBO dipelajari secara bertahap melalui satu studi kasus yang sama, yaitu sistem perbankan mini berbasis console dengan bahasa Java. Setiap pertemuan menambahkan satu konsep baru: class dan object, asosiasi antar class, enkapsulasi, pewarisan (inheritance), dan polimorfisme.

2. Tujuan

Tujuan praktikum ini adalah memahami dasar-dasar PBO menggunakan bahasa Java, mulai dari pembuatan class, properti, dan method, hingga penerapan asosiasi, enkapsulasi, inheritance, dan polimorfisme pada sistem perbankan mini.

3. Manfaat

Laporan ini bermanfaat sebagai dokumentasi kegiatan praktikum dan media pembelajaran, khususnya pada mata kuliah Pemrograman Berorientasi Objek.

BAB 2
PEMBAHASAN

A. PRAKTIKUM 1

Praktikum 1 bertema pembuatan rekening sederhana: class memiliki properti untuk menyimpan data rekening dan method untuk mengolah data tersebut.

Class *Rekening* memodelkan sebuah rekening dengan properti nomor rekening, nama pemilik, dan saldo. Constructor `Rekening(nomor, nama, saldoAwal)` mengisi data awal. Method yang dibuat adalah `setorTunai()` (nominal harus lebih dari 0), `tariktunai()` (minimal Rp10.000 dan tidak boleh melebihi saldo), dan `cekInformasi()`.

Class *Main* adalah program utama berbasis CLI. Rekening disimpan dalam `ArrayList<Rekening>`, sedangkan variabel `akunAktif` menunjuk rekening yang sedang dipakai (bernilai `null` sebelum ada rekening). Menu dijalankan dengan `while` dan `switch`: buka rekening baru, setor tunai, tarik tunai, cek informasi, ganti akun, dan keluar. Tampilan program saat dijalankan adalah sebagai berikut.

[Tempel screenshot: hasil run Praktikum 1 (menu utama dan pembuatan rekening)]

B. PRAKTIKUM 2

Praktikum 2 membuat mekanisme riwayat transaksi, sehingga setiap rekening menyimpan histori transaksinya sendiri. Untuk itu dibuat class baru *Transaksi* dengan properti `idTransaksi`, `jenis`, dan `nominal`, serta method `cetakDetail()`.

Hubungan kedua class adalah asosiasi one-to-many: satu *Rekening* memiliki banyak *Transaksi* yang disimpan dalam `ArrayList<Transaksi> riwayatTransaksi`. List ini wajib diinisialisasi di constructor agar tidak terjadi `NullPointerException`. Setiap setor tunai dan tarik tunai membuat objek *Transaksi* baru dengan ID dari `System.currentTimeMillis()`, seperti pada kode berikut.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println(" Setor tunai Rp " + nominal
            + " Berhasil. Saldo saat ini: Rp " + saldo);
    } else {
        System.out.println(" Gagal: Nominal setor harus lebih dari 0! ");
    }
}
```

Riwayat transaksi dapat ditampilkan melalui menu Cetak Mutasi (`cetakMutasi()`), seperti pada gambar berikut.

[Tempel screenshot: hasil Cetak Mutasi pada Praktikum 2]

C. PRAKTIKUM 3

Praktikum 3 menerapkan konsep enkapsulasi pada program sebelumnya. Seluruh properti pada class *Rekening* dan *Transaksi* diubah menjadi `private` sehingga tidak dapat diakses langsung oleh class lain. Class lain hanya dapat mengakses data lewat method getter yang disediakan.

```
private String nomorRekening;
private String namaPemilik;
private double saldo;
private String pin;   // data sensitif

public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik()   { return namaPemilik; }

public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}
```

Pada class *Rekening* ditambahkan properti `pin` sebagai pengaman. PIN divalidasi harus 6 digit (di constructor dan di *Main* dengan regex `\\d{6}`), dan method `otentikasi()` dipakai untuk memeriksa PIN sebelum tarik tunai dan cetak mutasi. Class *Transaksi* hanya diberi getter tanpa setter, sehingga datanya bersifat baca-saja.

Mengapa tidak dibuat method setSaldo() dan getPin()?

Method `setSaldo()` tidak diperlukan karena perubahan saldo sudah dilakukan lewat `setorTunai()` dan `tariktunai()` yang memiliki validasi. Jika ada setter, saldo bisa diubah sembarangan tanpa aturan. Method `getPin()` juga tidak dibuat karena PIN adalah data rahasia. Bila class lain bisa membaca PIN, pengamanan menggunakan PIN menjadi tidak berarti. Pemeriksaan PIN cukup dilakukan lewat `otentikasi()` yang hanya mengembalikan benar atau salah.

[Tempel screenshot: hasil Praktikum 3 (permintaan PIN dan pesan akses ditolak jika PIN salah)]

D. PRAKTIKUM 4

Praktikum 4 menerapkan inheritance (pewarisan). Class *Rekening* menjadi class induk, dan dibuat dua class turunan, yaitu *RekeningTabungan* dan *RekeningGiro*, yang mewarisi properti dan method induknya.

Pada class induk, properti `saldo` dan `riwayatTransaksi` diubah menjadi `protected` agar dapat diakses class turunan, sedangkan `nomorRekening`, `namaPemilik`, dan `pin` tetap `private`. Dengan begitu class lain yang bukan turunan tetap tidak bisa mengaksesnya.

```
protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;
```

Class *RekeningTabungan* memiliki properti tambahan `sukuBunga` dan method `tambahBungaAkhirBulan()` untuk menghitung bunga, menambahkannya ke saldo, lalu mencatatnya sebagai transaksi berjenis “Bunga”. Constructor turunan memanggil constructor induk dengan `super(...)` pada baris pertama.

```
public class RekeningTabungan extends Rekening {
    private double sukuBunga;

    public RekeningTabungan(String nomor, String nama,
            double saldoAwal, String pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
    }
}
```

Class *RekeningGiro* memiliki properti `batasOverdraft` dan method `getBatasOverdraft()` yang mengembalikan nilainya. Pada *Main*, pengguna kini memilih produk saat membuka rekening, dan menu Simulasi Akhir Bulan memakai `instanceof` agar hanya berlaku untuk rekening tabungan.

[Tempel screenshot: hasil Praktikum 4 (pembuatan rekening tabungan/giro dan simulasi akhir bulan)]

E. PRAKTIKUM 5

Praktikum 5 berfokus pada polimorfisme, yaitu kemampuan satu pemanggilan method menghasilkan perilaku berbeda sesuai jenis objek yang sebenarnya. Konsep ini membuat program lebih fleksibel dan mudah dikembangkan.

Kasus yang diselesaikan adalah *RekeningGiro* yang memiliki aturan tarik tunai sendiri, berbeda dengan *Rekening* biasa. Giro boleh menarik melebihi saldo sampai batas overdraft, sehingga saldo bisa negatif. Untuk itu digunakan konsep overriding.

Langkah pertama adalah mengubah method `tarikTunai` di class *Rekening* dari `void` menjadi `boolean`, agar pemanggilnya tahu apakah penarikan berhasil.

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo
Anda: Rp" + saldo);
        return false;
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Tarik Tunai", nominal));
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa
saldo: Rp" + saldo);
        return true;
    }
}
```

Langkah kedua adalah melakukan overriding method `tarikTunai` pada class *RekeningGiro*. Penarikan diizinkan selama nominal tidak melebihi `saldo + batasOverdraft`.

```
@Override
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
        return false;
    } else if (nominal > (saldo + batasOverdraft)) {
        System.out.println("Transaksi Gagal: Melebihi batas overdraft!
Maksimal penarikan: Rp"
                + (saldo + batasOverdraft));
        return false;
    } else {
        saldo -= nominal;
        String idTrx = "TRX-G-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));
        System.out.println("Tarik tunai Giro Rp" + nominal
                + " berhasil. Saldo saat ini: Rp" + saldo);
        return true;
    }
}
```

Saat dijalankan, program mengizinkan saldo bernilai negatif pada RekeningGiro selama masih dalam batas overdraft, tetapi menolak penarikan yang melebihi saldo pada Rekening biasa (RekeningTabungan).

[Tempel screenshot: hasil run penarikan pada RekeningGiro (saldo menjadi negatif)]

Latihan Praktikum 5

1. **Membuat method transfer.** Pada class *Rekening* ditambahkan method `public void transfer(Rekening tujuan, double nominal, String pinInput)`. Parameter `tujuan` bersifat polimorfik karena dapat berupa RekeningTabungan maupun RekeningGiro. Logikanya: (i) lakukan `otentikasi(pinInput)` dan batalkan jika salah; (ii) panggil `this.tarikTunai(nominal)` (jika pengirim adalah Giro, aturan overdraft otomatis dipakai); (iii) jika hasilnya `true`, panggil `tujuan.setorTunai(nominal)`.

Pengerjaan

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    // 1. Autentikasi PIN
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer gagal: PIN salah!");
        return;
    }
    // 2. Tarik saldo dari rekening pengirim (polimorfisme)
    boolean berhasil = this.tarikTunai(nominal);

    // 3. Jika penarikan berhasil, setor ke rekening tujuan
    if (berhasil) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    } else {
        System.out.println("Transfer gagal!");
    }
}
```

2. **Integrasi ke CLI utama.** Pada *Main* ditambahkan menu “8. Transfer ke Rekening Lain”. Program meminta nomor rekening tujuan, mencarinya di `daftarRekening`, menolak jika tidak ditemukan atau sama dengan akun aktif, lalu meminta nominal dan PIN dan memanggil `akunAktif.transfer(akunTujuan, nominal, pin)`.

Pengerjaan

```
case 8:
    System.out.print("Masukkan Nomor Rekening Tujuan: ");
    String nomorTujuan = input.nextLine();
    Rekening akunTujuan = null;
    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(nomorTujuan)) {
            akunTujuan = rekening;
            break;
        }
```

```
    }
    if (akunTujuan == null) {
        System.out.println("Rekening tujuan tidak ditemukan!");
        break;
    }
    if (akunTujuan == akunAktif) {
        System.out.println("Tidak dapat mentransfer ke rekening sendiri!");
        break;
    }
    System.out.print("Masukkan nominal transfer: Rp");
    double nominal = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan PIN Anda: ");
    String pin1 = input.nextLine();
    akunAktif.transfer(akunTujuan, nominal, pin1);
    break;
```

Hasil Pengujian

Pengujian dilakukan dengan dua akun: akun pertama bernama Budi (Tabungan Umum, nomor 1001, saldo Rp5.000.000, suku bunga 2%) dan akun kedua bernama Siti (Giro Bisnis, nomor 1002, saldo Rp1.000.000, batas overdraft Rp500.000).

Pertama, akun Budi dibuat sebagai Tabungan Umum.

[Tempel screenshot: pembuatan akun Budi (Tabungan Umum)]

Kedua, akun Siti dibuat sebagai Giro Bisnis.

[Tempel screenshot: pembuatan akun Siti (Giro Bisnis)]

Ketiga, pada akun Siti dilakukan penarikan Rp1.200.000. Nominal ini melebihi saldo, tetapi masih dalam batas overdraft sehingga berhasil dan saldo menjadi negatif. Penarikan berikutnya sebesar Rp600.000 ditolak karena melebihi batas overdraft.

Hasil yang diharapkan:

```
Tarik tunai Giro Rp1200000.0 berhasil. Saldo saat ini: Rp-200000.0
Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp300000.0
```

[Tempel screenshot: penarikan pada akun Giro (berhasil dan ditolak)]

Keempat, pada akun Budi (Tabungan) dilakukan penarikan Rp6.000.000. Karena melebihi saldo dan Tabungan tidak memiliki overdraft, penarikan ditolak. Ini menunjukkan polimorfisme: pemanggilan tarikTunai yang sama menghasilkan perilaku berbeda pada Giro dan Tabungan.

Hasil yang diharapkan:

```
Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp5000000.0
```

Kelima, dari akun Budi dilakukan transfer Rp1.000.000 ke akun Siti (nomor 1002) menggunakan menu 8. Saldo Budi berkurang menjadi Rp4.000.000, sedangkan saldo Siti bertambah dari Rp-200.000 menjadi Rp800.000.

Hasil yang diharapkan:

```
Tarik tunai Rp1000000.0 berhasil. Sisa saldo: Rp4000000.0
 Setor tunai Rp 1000000.0 Berhasil. Saldo saat ini: Rp 800000.0
Transfer berhasil!
```

[Tempel screenshot: transfer dari akun Budi ke akun Siti]

Keenam, simulasi akhir bulan (menu 7) pada akun Budi menambahkan bunga 2% dari saldo Rp4.000.000, yaitu Rp80.000. Bila menu yang sama dipilih pada akun Giro, program menolak karena fitur bunga hanya untuk Rekening Tabungan.

Hasil yang diharapkan:

```
Bunga 2.0% berhasil ditambahkan: Rp80000.0
```

Kesimpulan Praktikum 5

Polimorfisme melalui method overriding membuat RekeningGiro memiliki aturan tarik tunai sendiri tanpa mengubah kode di class Rekening maupun Main. Method transfer cukup memanggil tarikTunai, dan Java otomatis memilih versi yang sesuai dengan jenis rekening pengirim. Nilai kembalian boolean pada tarikTunai memastikan uang hanya disetor ke rekening tujuan jika penarikan berhasil.

F. GITHUB DAN VIDEO PENJELASAN

Kode program dapat dilihat pada repository GitHub melalui link di bawah ini:

https://github.com/nofriilham/PrakPBO_C_2511531013

Video penjelasan berupa video YouTube yang dapat diakses melalui link di bawah ini:

[Tempel link video YouTube di sini]